Criterion E – Evaluation

Section Overview

Product Evaluation 2
Tabular Evaluation 2
Client Comments 3
Evaluation 3
Improvements & Extensions Based on Client Feedback 3

Product Evaluation

Tabular Evaluation

Success Criteria	Met?	Client Comments
Intuitive and minimalistic single-window layout displaying coronavirus data quantitatively and qualitatively.	Yes	"The layout and presentation is very convenient for me because the data I need is all in one place."
Presentation of four types of daily and national coronavirus data. Total cases, new cases, total deaths, and new deaths.	Yes	"I can easily see the four types of data and all of which are daily and national values."
Colour-coded, titled, and date-labelled map showing the severity of coronavirus in each country.	Yes	"The map is well labeled, has a title, and is color-coded which easily allows me to get a general sense of the severity of every country."
The map should update based on a change in date.	Yes	"I can slide to a new date or auto-play and the map changes accordingly."
Date changing should be quick and automatable.	Yes	"I indeed can quickly change date and let it change by itself."
The map should display all four types of data individually and updates accordingly.	Yes	"The map updates and displays all four types of data individually when I change the type selection."
Program contains a ranking that shows the top 20 most severe countries of a data type.	Yes	"The top 20 rankings is very useful for me to quickly see which countries are most severe."
This ranking should be titled and labelled in terms of which of the four types of data is currently on display.	Yes	"I can easily find the rankings because it is titled and I can also clearly see which type of data the rankings is for."
For each entry in the ranking, there should be three pieces of information displayed. Country name. Statistical value corresponding to the currently displayed data. Bar that represents the relative severity between countries in the top 20 rankings.	Yes	"The rankings has a country name, a bar, and a value. The bar is especially helpful for me to get a general idea of how the countries compare."
More details about a country should be accessible should the user wish to look more into a country. Specifically, there should be a graph of the all-time historical values of a data type for a country.	Yes	"There is a graph with the historical values for a country for a data type."
There should be some indication of where the current day on the graph is.	Yes	"I can clearly see where the current day on the graph from the circle."
There should also be a quantitative aspect in which the numerical data for a country is presented in a table.	Yes	"Very good. I like this feature because I can find out more about a country to substantiate my claims when making an educated suggestion."
Like the graph, there should be some indication of the present-day value in the table.	Yes	"The table automatically highlights the current day."
As a result, there should be a place to search for a country by its name.	Yes	"Yes. I can search for a country by its name, although it would be helpful to have a autocomplete feature."

Table E.1: evaluation of product, based on client feedback, against the success criteria.

Client Comments

"This project is very impressive, and I thank you for your hard work. It has already alleviated a lot of my pressure, so I am grateful for your diligence in making this program for me. However, some areas can be better. As I mentioned, it would be helpful to have an autocomplete feature when I search for a country. It would be cool if there's some sort of AI feature where the program can suggest some countries for me. I think this would be an interesting feature because it not only provides me with a reference point for my recommendations, but it also interests me to see a computer's decision. Furthermore, I often work late at night so sometimes this robot may be more reliable than me! Again, many thanks for this program, I wish you luck on your schoolwork!"

Evaluation

I met all the success criteria as shown in the table above. This product effectively solved the client's troubles and can be further improved and extended referenced in Criterion C, D and below.

Improvements & Extensions Based on Client Feedback

- Client request: autocomplete in country search.
    - Benefits:
        - Easy searching for client.
        - Client need not have perfect spelling.
        - Faster than typing entire country.
    - Implementation (proof of realistic): TreeMap data has format <date, <country, RecordItem>> where countries are in alphabetic order. Compare JTextField String with inner TreeMap Key to find first String with full match.
        - Find full match using String.startsWith().
        - Find using regular expression i.e., regex.Pattern.compile.("^"+countryName)…
- Client request: AI country suggestion.
    - Benefits:
        - Consistent decision making (no sense of fatigue).
        - Provides frame of reference to client.
    - Implementation 1 – Dijkstra's Algorithm
        - Functionality: make decision based on route between destinations and covid severity.
        - Benefits: provides frame of reference so the client can compare his results with the program's results and perhaps learn from it.
        - Overview: Dijkstra's algorithm finds the shortest positive-weighted path between two nodes. Since some flights ought to stop at an intermediate country, there is a need to choose an optimal path.
            - Starting node: client's customer's current location.
            - Ending Node: their destination.
            - Weight: distance between locations.
                - Merge covid severity to the weight.
        - Data structure:
            - Adjacency list: array of nodes containing a list of destinations, storing (from, to, weight) in each list element.
            - Priority queue: queue with priorities to store the next node to visit.
            - See Appendix D – Dijkstra Algorithm.
    - Implementation 2 – Neural Networks
        - Functionality: give client suggestions based on the client's decision-making style after sufficient data training.

- Benefits: mimics client selection style, allowing for consistency.
- Overview: in logistic regression gradient descent, we train a custom set of adjustment(b) and weight(w) factors with testing data to predict outcomes of future data sets.
    - A single neuron has equation $z = w_1x_1 + w_2x_2 + \cdots + w_nx_n + b$.
        - x: feature variable.
            - Factors in the decision making, e.g.,
                - Covid situation.
                - Flight stops.
                - Flight, hotel expenses.
                - Destination activity fee. Etc.
        - w: weight of each feature variable.
        - b: final adjustment factor.
    - Forward propagation – compute loss:
        - Modify weights(w) and adjustments(b) to minimize a loss function.
    - Backward propagation – compute derivatives (gradient descent):
        - Find derivatives in each step of forward propagation to determine how much "w" and "b" should change by.
- Language: Python for its NumPy module to perform matrix operations.
- See Appendix E – Machine Learning Logistic Regression.

(*409 words; 33 Evaluation + 376 Improvements & Extensions Based on Client Feedback*)